KlimateX System Guide

How the platform works, end to end · Access Youth Tech Forge 2026 · Team KlimateX (team20)

Version 0.2 · 9 November 2026

1. **What KlimateX is**
   - 1.1 Who uses it
   - 1.2 What is in this package
2. **The system at a glance**
3. **A home's journey through the system**
4. **The carbon maths**
   - 4.1 Method
   - 4.2 Worked example: one home, one month
   - 4.3 Placeholders to confirm before any real filing
5. **The nine checks and the trust score**
   - 5.1 The two planted frauds in the demo
6. **Monitoring periods and credits**
7. **How the money moves**
   - 7.1 The split (illustrative)
   - 7.2 Safety rules in the money code
8. **The ledger and the verify page**
9. **The data model**
10. **API reference**
    - 10.1 System
    - 10.2 Household app
    - 10.3 Auditor portal
    - 10.4 Buyer marketplace
    - 10.5 Public and webhooks
11. **Connecting to Access's sandbox**
12. **The demo, step by step**
    - 12.1 The three-minute stage demo
13. **Running it on a laptop**
14. **Deploying to team20**
15. **Troubleshooting**
16. **What is real, what is simulated, what is still open**
17. **Glossary**

1. What KlimateX is

A Lagos home that swaps its petrol generator for solar stops burning about 1,300 litres of fuel a year, close to 2 tonnes of CO_2 once grid hours are taken out. Companies pay for proven tonnes, but proving one home's saving costs more than the saving is worth, so the family never sees that money.

KlimateX reads the saving straight off the home's solar inverter, checks it nine ways, groups thousands of homes into one project so an auditor can sign it off cheaply, turns each tonne into a serial-numbered credit, sells it to a company, and pays the family's share through Access Bank: into its wallet or straight off its solar loan.

Every part of the system serves one idea: **make the proof of a home's saving nearly free, so the money can reach the family.**

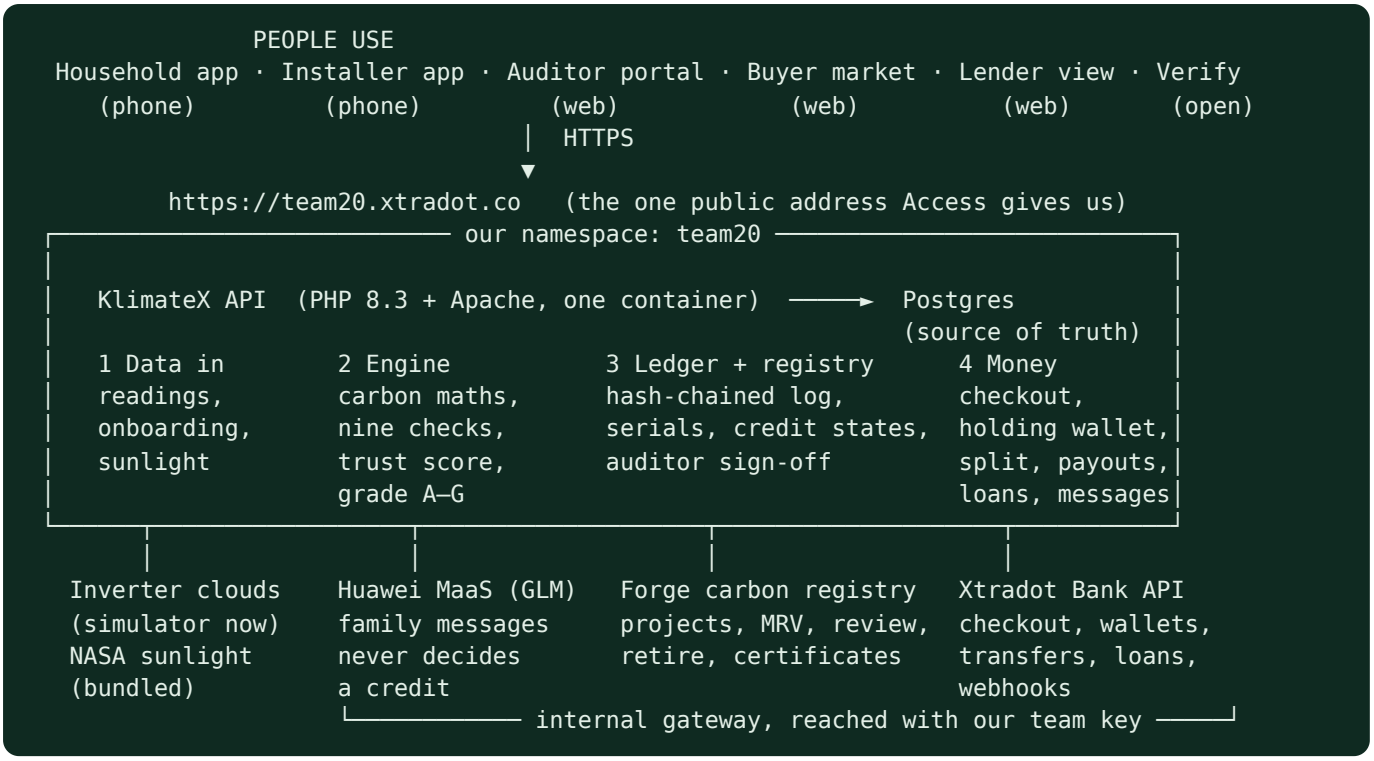
1.1 Who uses it

USER	WHAT THEY DO IN KLIMATEX	WHERE
Family (household)	See their grade, litres saved, payouts and certificate	Household app (phone)
Installer	Onboard homes, scrap the old generator, watch their fleet	Installer app (phone)
Auditor	Review flagged homes, draw a sample, sign off a period	Auditor portal (web)
Credit buyer	Buy and retire credits, download the certificate	Buyer marketplace (web)
Access Bank	Watch homes and loans by grade, payouts, deposits	Lender dashboard (web)
Anyone	Check a credit serial or a home certificate	Public verify page
KlimateX staff	Watch flags, splits and payout reconciliation	Operations console

1.2 What is in this package

FOLDER	CONTENTS
`backend-php/`	**The backend we deploy.** Plain PHP 8.3, PDO, curl. Dockerfile and Kubernetes files for team20
`backend-python/`	The same backend in FastAPI. Same routes and results. Kept as a reference and fallback
`docs/SYSTEM_GUIDE.md` / `.pdf`	This guide
`docs/KlimateX_Pitch_Brief.pdf`	The 11-page pitch brief
`docs/api-requests.http`	Every API call, ready to run in VS Code (REST Client) or JetBrains

2. The system at a glance



Three facts decide this shape:

1. **Access gives each team one space on its Kubernetes cluster (`team20`) and one public address.** We deploy our container there and it appears at `https://team20.xtradot.co`. The website is served from the same container at `/app`.
2. **The bank, the carbon registry and the Huawei models only answer calls from inside the cluster.** So the backend must run in `team20`. Phones and browsers never call the bank directly; they call our backend, and our backend calls the bank. That is also why the team key never sits on a phone.
3. **The sandbox keeps bank data in memory and can be reset.** Our Postgres database is the record of truth.

3. A home's journey through the system

This is the life of one home, with the part of the system that handles each step.

#	STEP	WHAT HAPPENS	HANDLED BY
1	**Grade**	The home is graded A–G by how much of its power comes from a generator. Before solar most Lagos homes are F. The certificate shows the grade it could reach and the Access loan to get there	`GET /api/homes/{id}/certificate`

#	STEP	WHAT HAPPENS	HANDLED BY
2	**Join**	The family e-signs consent for KlimateX to sell its carbon savings. The installer records the generator's serial and nameplate, takes it as a trade-in, and uploads the recycler's scrap certificate	Installer app → `households`, `generators`
3	**Read**	The inverter reports daily: kWh generated, solar delivered to the home, grid kWh, backup generator kWh. One inverter serial, one home	Inverter adapter → `readings`
4	**Count**	Each month, the engine turns metered solar into tonnes using Gold Standard's generator equations (section 4)	`POST /api/periods/{id}/compute`
5	**Check**	Nine checks give a trust score. 80+ can earn credits; 50–79 is held for the auditor; under 50 is excluded (section 5)	Same call
6	**Audit**	Every 6–12 months the auditor gets a sample drawn with a visible seed, always including held homes. They review the evidence and sign off	`/sample`, `/sites/{id}/review`, `/signoff`
7	**Issue**	Whole tonnes become serial-numbered credits (`KX-NG-2026-LAG01-000001`). Fractions carry to the next period. The period is filed with the carbon registry	Sign-off → `credits`, registry
8	**Sell**	A company reserves credits for 15 minutes and pays on Access's hosted checkout	`POST /api/orders`
9	**Retire**	Only after the payment is verified are the credits retired in the buyer's name. They can never be sold again	`POST /api/orders/{id}/confirm`
10	**Split**	The money is split: 50% families (by tonnes each home earned), 15% installer, 20% audit and registry, 15% KlimateX	Same call
11	**Pay**	Families are paid into their Access wallet or straight off their solar loan. The installer is paid by bank transfer after a name check	Bank client
12	**Prove**	Every step is written to a hash-chained ledger. Anyone can look up a serial on the verify page	`GET /api/verify/{serial}`

4. The carbon maths

4.1 Method

KlimateX follows **Gold Standard JUST: Fossil Fuel Generators v1.0** (March 2026), *Scenario B: unreliable grid*. Most Lagos homes get a few hours of grid power a day, so solar replaces partly generator power and partly grid power. Only the generator part counts at the full petrol rate.

For each home, each month:

SYMBOL	MEANING	HOW WE GET IT
EG_RES	Solar delivered to the home (kWh)	Sum of daily `kwh_solar_to_load`, **excluding failed days**
EG_BAS cap	Most the old generator could have produced	rated kVA × 0.8 × 24 h × days × 0.9 availability
S_grid	Share of the baseline that was grid	grid hours a day ÷ 24 (onboarding survey)
EF_BL	Generator emission factor	**1.59 t/MWh** (0.7 L/kWh × 2.27 kg CO_2/L) · placeholder
EF_grid	Grid emission factor	**0.40 t/MWh** · placeholder
DAF	Net-zero adjustment	**5%** · placeholder

```
BE  = min(EG_RES, EG_BAS) × (S_genset × EF_BL + S_grid × EF_grid) × (1 − DAF)    baseline
emissions
AE  = backup generator kWh × EF_BL + 0.01 t embodied                             project
emissions
LE  = max(0, (baseline kWh − EG_RES) × 1.3 t/MWh)                                leakage
penalty
ER  = max(0, BE − AE − LE)                                                       tonnes
credited
```

4.2 Worked example: one home, one month

152 kWh of solar to the home, grid 6 hours a day (so 75% generator, 25% grid), no backup use:

```
0.152 MWh × (0.75 × 1.59 + 0.25 × 0.40) = 0.152 × 1.2925 = 0.1965 t
× (1 − 0.05)                                              = 0.1866 t
− 0.01 t embodied                                         = 0.177 t
```

About **0.18 t a month, 2.1 t a year**. Our first estimate was 2.9 t; it ignored grid hours. The test suite checks this exact example.

4.3 Placeholders to confirm before any real filing

The factors 1.59 and 0.40 t/MWh, the 5% adjustment and the 1.3 t/MWh leakage rate are in `src/Config.php`. Replace them with the values in the methodology's annexes before anything is filed for real. Every result stores the factors it used.

5. The nine checks and the trust score

Rules first, so every result can be explained to an auditor. Days that fail earn zero. Missing days count as zero. Nothing is ever filled upward. When in doubt, we credit less.

#	CHECK	RULE	IF IT FAILS
1	**Sunlight**	Daily kWh ≤ NASA irradiance × panel kWp × 1.1	That day earns zero
2	**Energy balance**	Solar to the home ≤ solar generated (2% tolerance)	Flag
3	**Clone**	Daily profile correlates above 0.99 with another inverter over 14+ days	Both homes held
4	**Flat line**	A 7-day window with under 1% variation	Flag
5	**Spike**	A day over 3× the trailing 30-day median	That day earns zero
6	**Missing data**	No reading for a day	Counts as zero
7	**Identity**	Serials unique (database rule); install photo within 200 m of the address	Home blocked
8	**Scrapped**	Scrap date and recycler certificate on file	Tonnes held
9	**Backup share**	Backup generator ≤ 10% of the home's power	Excluded for the period

Trust score starts at 100 and loses points: up to 60 for failed days (by share of the period), 40 for a clone match, 30 for a flat line, 20 for an energy-balance break, up to 20 for missing days. A GPS mismatch sets it to 0.

SCORE	STATUS	MEANING
80–100 and generator scrapped	`eligible`	Earns credits
50–79, or generator not yet scrapped	`held`	Earns nothing until the auditor clears it; always in the sample
Under 50, or backup over 10%	`excluded`	Earns nothing; the family is told why

5.1 The two planted frauds in the demo

- **Home 17** reports about three times what the sun allowed every day. The sunlight check fails every day, score 40, **excluded**, zero tonnes.
- **Home 19** replays Home 04's data stream. The clone check matches them, both drop to 60 and are **held**. The auditor keeps both held pending a site visit, because the data alone cannot tell which is genuine.

6. Monitoring periods and credits

A **monitoring period** is the unit an auditor signs: typically 6–12 months for a batch of homes. Its states:

```
open ──compute──► computed ──sample──► sampled ──review sampled homes──► signoff ──► signed
```

- **Compute** runs the maths and checks for every home and stores a `site_results` row with tonnes per month, the checks and their evidence, trust score, status and grade.
- **Sample** always includes every held home, plus a random 20% of eligible homes. The seed is recorded (`seed=42` in the demo), so anyone can redraw the same sample and check it was not cherry-picked.
- **Review** each sampled home: `ok`, `clear` (held → eligible), `keep_held`, or `exclude`. Sign-off is refused until every sampled home is reviewed.
- **Sign-off** files the period with the carbon registry, then mints one credit per whole tonne of eligible tonnes. The remainder carries forward.

How a credit is built. Each credit is an equal slice of every eligible home-month in the period. A credit's `contributions` list says how much of that tonne came from each home in each month, and always sums to 1.000 t. Because every credit holds a slice of every home, **whichever credits sell, every family is paid in proportion to the tonnes it earned.** (An earlier version filled credits home by home, which paid only the first few homes. That was unfair and was changed.)

Serial format: `KX-NG-{vintage}-{project}-{sequence}`, for example `KX-NG-2026-LAG01-000001`.

Credit states: `listed` → `reserved` (15 minutes, during checkout) → `retired` (in the buyer's name, final). An unpaid reservation is released automatically.

7. How the money moves

```
1 PAY              2 HOLD                    3 RETIRE                  4 SPLIT + PAY
Buyer pays   ──►   Access holding wallet ──► credits retired in   ──►  ledger splits by tonnes,
at checkout        (funds wait here)         the buyer's name          pays wallets, loans,
installer
```

The order of these steps is the most important rule in the money flow: **pay, confirm, retire, then release.** Credits are never retired before payment is verified, and no one is paid before the credits are retired.

7.1 The split (illustrative)

Price: ₦15,000 a credit (about $10). Demo order: 10 credits, ₦150,000.

SHARE	%	DEMO AMOUNT	PAID TO	HOW
Families	50%	₦75,000	17 households, by tonnes earned	Wallet transfer; or wallet then loan repayment if the home has a solar loan
Installer	15%	₦22,500	Arewa Solar Co-operative Society	Name enquiry, then interbank transfer

SHARE	%	DEMO AMOUNT	PAID TO	HOW
Audit and registry	20%	₦30,000	Reserve held by KlimateX	Booked in our ledger
KlimateX	15%	₦22,500	Platform fee	Booked; also takes any leftover kobo

Money is always whole kobo. The household pot is divided by tonnes; any kobo left from rounding goes to the home with the most tonnes, so the total always matches to the kobo. Government fees, if credits are authorised for export, come off before the split.

7.2 Safety rules in the money code

- **Verify before value.** Every order is confirmed with `GET /payments/{ref}/verify` before credits are retired.
- **Idempotency key on every money call.** The key is built from the order and recipient, so running a payout twice can never pay twice. The test suite confirms an order twice and checks the total is still ₦150,000.
- **Name enquiry before every transfer.** The installer's resolved account name is stored with the payout.
- **On a transfer timeout (504), never resend.** The client requeries `GET /transfers/{reference}` until the result is final. Until then the payout stays `pending`, never `paid`.
- **Back off on 429.** The sandbox allows 60 requests a minute shared across the bank and the AI models.
- **Webhooks are signed.** In live mode the HMAC is checked and anything older than 5 minutes is rejected. Each event is stored once by its id.

8. The ledger and the verify page

Every important event (seed, period computed, sampled, site reviewed, signed, order reserved, credits retired, each payout, order split) is appended to the `ledger` table. Each entry stores:

```
hash = SHA-256( previous entry's hash + canonical JSON of {type, payload, timestamp} )
```

Change any past entry and every later hash breaks. `GET /api/ledger/verify` walks the whole chain and reports `ok: true` with the latest hash (the root), or the first broken entry. The test suite edits one row on purpose and confirms the break is caught. In the pilot, the day's root is published on the verify page so outsiders can check it.

We do not put credits on a blockchain or turn them into tokens. Hash-chaining gives tamper evidence without creating a second tradable asset, which avoids double-counting questions.

The public verify page (`GET /api/verify/{serial}`) shows a credit's vintage, project, status, the auditor and sign-off time, who retired it and when, and the home-months behind it. Homes are pseudonymised (`home-012`), never named.

9. The data model

Money in kobo (integers). JSON fields stored as text. The chain that must never break:

```
household → site → inverter → reading → period → site_result → credit → order → payout
```

TABLE	KEY FIELDS	NOTES
`installers`	name, accreditation_id, account_number, bank_code	Paid by interbank transfer
`households`	name, wallet_id, consent_hash, loan_id	Synthetic in the sandbox
`sites`	label, area, lat/lon, install_lat/lon, grid_hours_per_day, baseline_kwh_per_day, panel_kwp	One home
`generators`	serial (unique), rated_kva, scrapped_on, scrap_certificate, trade_in_value_kobo	The old generator
`inverters`	serial (unique), site_id (unique), vendor	One inverter, one home
`readings`	inverter_id + day (unique), kwh_generated, kwh_solar_to_load, kwh_from_grid, kwh_from_backup, payload_hash	Daily
`sunlight`	area + day (unique), kwh_m2	NASA POWER or placeholder
`periods`	start_day, end_day, status, sample_seed, sample_site_ids, signed_by, total_t, carried_t, registry ids	Unit the auditor signs
`site_results`	period + site (unique), tco2e, monthly, trust_score, status, checks, generator_share, grade, reviewed	One home in one period
`credits`	serial (key), vintage, status, contributions, order_id, retired_for, retired_at, simulated	One tonne
`orders`	buyer_name, tonnes, amount_kobo, status, payment_reference (unique), expires_at, registry_certificate_id	One purchase
`payouts`	order_id, role, recipient, amount_kobo, idempotency_key (unique), reference (unique), status, bank_response	One payment
`ledger`	ts, event_type, payload, prev_hash, hash (unique)	Insert only

TABLE	KEY FIELDS	NOTES
`webhook_events`	event_id (key), event_type, body	De-duplication
`kv`	key, value	Holding wallet id, registry project id, mock bank state

In Postgres, revoke `UPDATE` and `DELETE` on `ledger` from the app's user for the pilot.

10. API reference

Base URL: `https://team20.xtradot.co` (live) or `http://localhost:8080` (laptop). All responses are JSON. Errors return `{"detail": "..."}` with 404, 409 (wrong state), 422 (bad input) or 502 (sandbox error). The PHP and Python backends expose the same routes.

10.1 System

METHOD AND PATH	PURPOSE
`GET /healthz`	`{"ok":true,"mode":"live","runtime":"php 8.3.x"}`. Used by the Kubernetes readiness probe
`GET /api/sandbox/check`	Proves the pod can reach the bank sandbox with the team key
`POST /api/sandbox/register-webhook`	Registers our webhook URL. The secret is returned **once**; store it as `WEBHOOK_SECRET`
`POST /api/demo/run`	The whole story in one call (section 12)

10.2 Household app

METHOD AND PATH	RETURNS
`GET /api/homes`	Every home with grade, trust score, status and tonnes for the latest period
`GET /api/homes/{id}`	One home: site, household, generator, inverter, daily readings, every period's results and checks
`GET /api/homes/{id}/certificate`	Home Energy Certificate data
`GET /api/homes/{id}/message?lang=en`	A plain-language monthly message. `lang`: `en`, `pcm` (Pidgin), `yo`, `ig`, `ha`. Written by Huawei GLM in live mode

Sample `GET /api/homes/12/certificate` (Adaeze):

```
        {
          "certificate": "KX-HEC-2026-LAG-000012", "home": "Home 12", "household": "Adaeze",
          "grade_before": "F", "generator_share_before": 0.75,
          "grade_now": "B", "generator_share_now": 0.0598, "potential": "A with a 2.4 kWh
battery",
          "credited_tco2e_period": 1.039, "credited_tco2e_per_year": 2.08,
          "petrol_litres_avoided_period": 526, "payouts_naira": 4275.76, "trust_score": 100,
          "checks": [{"name": "sunlight", "result": "pass"}, "... nine in all ..."],
          "scale": "KlimateX scale (proposed), not an official rating"
        }
```


10.3 Auditor portal

METHOD AND PATH	BODY OR QUERY	RETURNS
`POST /api/periods`	`{"start":"2026-01-01","end":"2026-06-30"}`	`{"period": 1}`
`POST /api/periods/{id}/compute`	none	Eligible tonnes and every home's tonnes, score and status
`GET /api/periods/{id}`	none	Status, sample, signer, and each home's tonnes, score, status, reviewed flag and failed checks
`POST /api/periods/{id}/sample?seed=42`	none	The sample and the held homes it always includes
`POST /api/periods/{id}/sites/{siteId}/review`	`{"decision":"ok","note":"evidence consistent"}`	The home's new status
`POST /api/periods/{id}/signoff?auditor=Name`	none	Credits minted, tonnes carried, first and last serial, registry ids

Sample sign-off from the demo:


```
        { "credits_minted": 18, "carried_t": 0.225,
          "first": "KX-NG-2026-LAG01-000001", "last": "KX-NG-2026-LAG01-000018",
          "registry_project": "prj_f95cc489", "registry_mrv": "mrv_f0e1e400" }
```


10.4 Buyer marketplace

METHOD AND PATH	BODY OR QUERY	RETURNS
`GET /api/credits?status=listed`	`status`: listed, reserved or retired	Count, price and serials

METHOD AND PATH	BODY OR QUERY	RETURNS
`POST /api/orders`	`{"buyer_name":"Mama Nkechi Provisions Ltd","tonnes":10}`	Order id, amount, `payment_url` (open it to pay), reference
`POST /api/orders/{id}/confirm`	none	Verifies payment, retires, splits, pays. Safe to call twice. Returns `"action":"waiting"` if unpaid
`GET /api/orders/{id}`	none	Status, certificate id, totals by role, every payout

Test card for the sandbox's hosted checkout: **5399 8300 0000 0008**, any future expiry, CVV 123, PIN 1234, OTP 123456.

10.5 Public and webhooks

METHOD AND PATH	RETURNS
`GET /api/verify/{serial}`	The credit's story, home-months pseudonymised, and the ledger root
`GET /api/ledger/verify`	`{"ok":true,"entries":34,"root":"bdea…"}`
`POST /hooks/forge`	Sandbox webhooks. On `payment.successful` it confirms the matching order

11. Connecting to Access's sandbox

SERVICE	ADDRESS INSIDE THE CLUSTER	AUTH
Xtradot Bank API and carbon registry	`http://bank-api.ingress-apisix.svc.cluster.local/v1`	header `x-api-key: $GATEWAY_KEY`
Huawei MaaS (OpenAI-compatible)	`http://huawei-maas.ingress-apisix.svc.cluster.local/openai/v1`	header `apikey: $GATEWAY_KEY`
Webhooks back to us	`http://app.team20.svc.cluster.local/hooks/forge`	HMAC in `x-forge-signature`

`GATEWAY_KEY` is already in our namespace as the `huawei-gateway` secret; `k8s/app.yaml` injects it. It never goes in code, images, Git or a phone.

Sandbox test accounts we use (synthetic, from the platform guide):

USE	ACCOUNT
KlimateX operating account (pays the installer)	0123456790
Installer: Arewa Solar Co-operative Society	0123987005, Zenith (000015)
Buyer	Mama Nkechi Provisions Ltd (pays by test card on checkout)
Household wallets	Created by the backend at seed time

Rules from the platform guide we must keep

- Synthetic data only in the sandbox: no real names, BVNs, NINs or account numbers.
- 60 requests a minute, shared by the whole team across the bank and the AI.
- Never resend a transfer after a 504; requery.
- Respond to webhooks with 2xx within 10 seconds.
- The sandbox can reset to its opening state. Our Postgres is the record.
- Kubeconfig and team key are revoked when the event closes. Export anything needed before then.

Request formats still to confirm. The interbank transfer and name enquiry follow the guide's own examples. The wallet, checkout, carbon registry and loan request bodies are best guesses, marked `TODO(spec)` in `src/Clients.php`. If the sandbox rejects one, its error message says which field it expected; change that one line.

12. The demo, step by step

`POST /api/demo/run` runs the whole story against whichever bank the backend is pointed at (mock or live):

1. **Seed** 20 Surulere homes with daily data from January to June 2026. Adaeze is Home 12.
2. **Compute** the period: 18.2 eligible tonnes from 17 homes; Home 17 excluded; Homes 04 and 19 held.
3. **Sample** with seed 42: homes 3, 4, 7, 9, 12 and 19 (the two held homes are always in).
4. **Review**: sampled eligible homes marked `ok`; the clone pair kept `held` for a site visit.
5. **Sign off**: 18 credits minted, 0.225 t carried to the next period.
6. **Sell** 10 credits to Mama Nkechi Provisions for ₦150,000; pay with the test card.
7. **Confirm**: payment verified, 10 credits retired, money split, 17 families and the installer paid.
8. **Ledger** checked: 34 entries, chain intact.

12.1 The three-minute stage demo

TIME	ON SCREEN	API BEHIND IT	THE JUDGE LEARNS
0:00	Adaeze's certificate: grade F to B, generator scrapped, loan offer	`/api/homes/12/certificate`	A real activity, graded
0:25	Daily solar against the sunlight ceiling; tonnes; equations one click away	`/api/homes/12`	Metered, Gold Standard maths
0:50	Home 17 fails the sunlight check; Home 19 caught copying Home 04	`/api/periods/1`	Fraud caught by the system

TIME	ON SCREEN	API BEHIND IT	THE JUDGE LEARNS
1:20	Auditor draws the sample with a visible seed, signs off, 18 serials minted	`/sample`, `/review`, `/signoff`	Independent, one serial a tonne
1:50	Buyer pays ₦150,000 on the sandbox, 10 credits retired, certificate opens	`/api/orders`, `/confirm`	Sold once, publicly checkable
2:30	The split runs; family wallets rise; Adaeze's goes straight to her loan	`/api/orders/1`	Every naira traced, through Access

Say on stage what is real and what is simulated. Real: the maths, the nine checks, the ledger, and the sandbox wallets, payouts and registry calls. Simulated: the inverter data, the households and installer, the auditor (a teammate) and Gold Standard issuance.

Demo safety: fixed seeds, so every run gives the same numbers; reset the sandbox (`POST /admin/reset`) and the database before the run; Postgres keeps the record if the sandbox restarts; keep a recorded backup video.

13. Running it on a laptop

```
cd backend-php
php -S 0.0.0.0:8080 -t public public/index.php      # mock mode, SQLite in /tmp
curl -X POST localhost:8080/api/demo/run
php tests/run.php                                    # 19 checks, all PASS
```

Mock mode needs no sandbox: a fake bank and registry run inside the app, with state kept in the database. To start fresh, delete `/tmp/klimatex.db` (or set `DB_DSN` to another file).

The 19 tests check: the worked example; Home 17 excluded; Homes 04 and 19 held; 17 eligible; at least 10 credits; order settled; ₦150,000 total; the exact split; payouts add up; 17 families paid; 10 credits retired; the verify page shows the buyer; each serial sums to 1.000 t; the ledger holds; confirming twice never pays twice; Adaeze F to B; Adaeze paid; the family message; tampering detected.

14. Deploying to team20

You need: the kubeconfig file `team20.yaml` (kept private), `kubectl` 1.30+, Docker, and a container registry account (GitHub Container Registry works).

```
export KUBECONFIG=~/forge/team20.yaml
kubectl get pods                                    # "No resources found" means
connected

# 1. Secrets (once)
kubectl create secret generic klimatex-db --from-literal=POSTGRES_PASSWORD='<strong-
password>'
kubectl create secret generic klimatex-secrets \
  --from-literal=DB_DSN='pgsql:host=postgres;port=5432;dbname=klimatex' \
  --from-literal=DB_USER='klimatex' --from-literal=DB_PASS='<strong-password>' \
  --from-literal=WEBHOOK_SECRET='pending'

# 2. Database
kubectl apply -f k8s/postgres.yaml && kubectl rollout status deployment/postgres

# 3. Image (the cluster's nodes are linux/amd64)
docker build --platform linux/amd64 -t ghcr.io/<you>/klimatex-php:v1 .
docker push ghcr.io/<you>/klimatex-php:v1

# 4. App: edit the image line in k8s/app.yaml, then
kubectl apply -f k8s/app.yaml && kubectl rollout status deployment/app
curl https://team20.xtradot.co/healthz

# 5. Bank check, webhook, secret
curl https://team20.xtradot.co/api/sandbox/check
curl -X POST https://team20.xtradot.co/api/sandbox/register-webhook
kubectl patch secret klimatex-secrets -p '{"stringData":{"WEBHOOK_SECRET":"
<secret>"}}'
kubectl rollout restart deployment/app

# 6. Demo against the real sandbox
curl -X POST https://team20.xtradot.co/api/demo/run
```

Day-to-day

TASK	COMMAND
Watch logs	`kubectl logs deploy/app -f`
Ship a new version	build and push `:v2`, then `kubectl set image deployment/app app=ghcr.io/<you>/klimatex-php:v2`
Undo a bad release	`kubectl rollout undo deployment/app`
See resource use	`kubectl describe resourcequota team-quota`
Open a database shell	`kubectl exec -it deploy/postgres -- psql -U klimatex`

Quota: the team may request 400m CPU and 768Mi memory in total, 10 pods, 5 services and one volume up to 10Gi. The app requests 150m/256Mi and Postgres 100m/256Mi, leaving room for a debug pod.

Adding the website: put the built files in `backend-php/public/app/` and rebuild the image. They appear at `https://team20.xtradot.co/app`. Use relative API calls (`/api/...`) so the same build works locally and live.

The APK: set its server address to `https://team20.xtradot.co`. It calls the household routes in section 10.2 and never calls the bank directly.

15. Troubleshooting

YOU SEE	LIKELY CAUSE	FIX
Pod stuck in `ImagePullBackOff`	Image is private or the name is wrong	Make it public, or create `regcred` and uncomment `imagePullSecrets`
`exec format error` in logs	Built on an ARM Mac without `--platform`	Rebuild with `--platform linux/amd64`
Pod `Pending`, "exceeded quota"	Requests above the team quota	Lower the requests in `k8s/*.yaml`; delete old debug pods
`CrashLoopBackOff`	Database not reachable or secret missing	`kubectl logs deploy/app`; check `klimatex-secrets` and that Postgres is ready
`/healthz` works but `/api/sandbox/check` gives 502	Team key missing or the gateway unreachable	`kubectl get secret huawei-gateway`; check `key_present` in the response
401 from the bank	Wrong or missing `x-api-key`	The `huawei-gateway` secret must be in `envFrom`
429 from the bank	Over 60 requests a minute across the team	The client backs off; avoid loops; reuse results
504 on a transfer	Outcome unknown	Handled: the client requeries. The payout stays `pending` until final
402 `INSUFFICIENT_FUNDS`	Holding wallet or operating account too low	Reset the sandbox or fund the wallet; the payout is marked `failed` and can be retried
400 or 422 naming a field	A `TODO(spec)` body differs from the real spec	Change that field in `src/Clients.php`
Webhook 401 "bad signature"	`WEBHOOK_SECRET` not set or stale	Re-register, patch the secret, restart the app
Order stays `retired`, not `settled`	One payout is `pending` or `failed`	`GET /api/orders/{id}`; call `/confirm` again; it never pays twice
Every number changed after a reset	The sandbox reset its data	Our Postgres still has everything; re-run the demo on a fresh database for the stage

16. What is real, what is simulated, what is still open

AREA	STATUS
Carbon maths, nine checks, trust score, grade	Built and tested

AREA	STATUS
Periods, sampling, sign-off, minting, pooled contributions	Built and tested
Orders, verification, retirement, split, idempotent payouts, 504 requery	Built and tested in mock mode
Ledger and verify endpoint	Built and tested, including tamper detection
Webhook HMAC and de-duplication	Built; needs the real secret in live mode
Interbank transfer and name enquiry	Match the guide's examples
Wallet, checkout, carbon registry, loan bodies	**Best guesses: confirm against the spec**
Inverter data	Simulated (20 homes, two planted frauds)
Sunlight	Placeholder series; run `php scripts/fetch_nasa.php` for real NASA POWER data
Emission factors and adjustments	Placeholders until confirmed from the JUST annexes
Auditor	A teammate in the demo; an accredited verifier in the pilot
Gold Standard issuance	Simulated through the sandbox registry; real issuance after the pilot audit
Website and APK	Next: build against the routes in section 10

17. Glossary

TERM	MEANING
Additionality	Proof that carbon money made the change happen. KlimateX funds new switches through trade-in and loans
Auditor (VVB)	An independent verifier who checks the data and signs a period
Carried tonnes	Fractions of a tonne left after minting whole credits, moved to the next period
Clone	Two inverters reporting the same data: one is probably copying the other
Credit	One tonne of CO_2 avoided, with a unique serial
Grade (A–G)	KlimateX's proposed scale: the metered share of a home's power still from a generator
Hash chain	A log where each entry includes the previous entry's hash, so any edit shows
Holding wallet	The Access wallet where a buyer's money waits until credits are retired
Idempotency key	A unique key on a money request so it can be safely retried without paying twice
JUST	Gold Standard's methodology for replacing fossil fuel generators with renewables
Kobo	One hundredth of a naira. All money in the system is whole kobo
Monitoring period	The stretch of time an auditor signs off, usually 6–12 months

TERM	MEANING
MRV	Measurement, reporting and verification
Name enquiry	A bank call that returns the account holder’s name before a transfer
Requery	Asking the bank for a transfer’s final status instead of sending it again
Retire	Use a credit to cover the buyer’s emissions. A retired credit can never be sold again
Scenario B	The JUST case where the grid is unreliable, so solar replaces both generator and grid power
Trust score	0–100 from the nine checks. 80+ earns credits